

Exercise 1: Real (Noisy) Data — Does the Line Still Fit Well? (Intermediate)

Goal: Understand what happens when data isn't perfectly linear — this is what real-world data looks like.

python

```
python

import numpy as np
np.random.seed(1)

area = np.array([50, 65, 70, 85, 100, 120, 140, 160, 180, 200, 90, 110, 130, 150, 75])
noise = np.random.normal(0, 3000, size=len(area)) # random noise
price = 300 * area + noise # same relationship as before, but noisy
```

Tasks:

1. Train a simple linear regression model (area $\rightarrow$ price) on this noisy data
2. Plot the scatter points and the best-fit line — notice the line no longer touches every point
3. Calculate MSE, MAE, and RMSE
4. Increase the noise (`np.random.normal(0, 10000, ...)`) and retrain — what happens to RMSE? Does the line change much?

Question to answer: Why doesn't the best-fit line pass through every point anymore? What does a higher RMSE tell you about the reliability of this model?

Exercise 3: Manual Gradient Descent (Advanced)

Goal: Implement gradient descent by hand (no sklearn) and confirm it matches `LinearRegression`.

Using the original clean dataset (area vs price):

python

```
python

area = np.array([50, 65, 70, 85, 100, 120, 140, 160, 180, 200])
price = np.array([15000, 19500, 21000, 25500, 30000, 36000, 42000, 48000, 54000, 60000])
```

Tasks:

1. Write your own gradient descent loop from scratch, following this pseudocode structure:

```
python

theta0, theta1 = 0, 0
alpha = 0.0001 # try a few different values
n = len(area)

for i in range(1000):
    y_pred = theta0 + theta1 * area
    error = y_pred - price

    d_theta0 = (1/n) * np.sum(error)
    d_theta1 = (1/n) * np.sum(error * area)

    theta0 = theta0 - alpha * d_theta0
    theta1 = theta1 - alpha * d_theta1

print("theta0:", theta0)
print("theta1:", theta1)
```

2. Compare your final theta0 and theta1 to model.intercept_ and model.coef_ from sklearn
3. Try changing alpha to something much larger (e.g., 0.01) — what happens? Does the model still converge?
4. Plot cost (J) vs iteration number to visualize the learning process

Question to answer: What happens to your theta values if alpha is too large? Why does that happen, based on what you learned about the learning rate?